Module 6: Generative AI with Large Language Models

Course Overview

Generative AI powered by Large Language Models (LLMs) represents one of the most transformative technological advances of our time. This module provides comprehensive training on building, fine-tuning, and deploying production-grade LLMs. Students will master the complete lifecycle of generative AI projects—from understanding foundational architectures and pre-training paradigms to implementing advanced fine-tuning techniques and alignment methods. This module bridges theory and practice, equipping graduates with industry-ready skills for developing cutting-edge AI solutions[1][2][3].

Table of Contents

1. Introduction to Large Language Models
2. Understanding LLM Architecture
3. Generative AI Project Lifecycle
4. LLM Pre-Training and Scaling
5. Fine-Tuning LLMs with Specific Instructions
6. Efficient Fine-Tuning of Parameters (PEFT)
7. Reinforcement Learning from Human Feedback (RLHF)
8. Advanced Alignment and Safety Techniques
9. Deployment and Inference Optimization
10. Hands-on Implementation Guide

---

1. Introduction to Large Language Models

1.1 What Are Large Language Models?

Large Language Models are neural networks with billions to trillions of parameters trained on massive amounts of text data to predict and generate human language. These models learn statistical patterns from diverse internet text, code, academic papers, and domain-specific data, enabling them to understand context, generate coherent text, answer questions, and perform reasoning tasks across diverse domains[4].

Key Characteristics of LLMs:

- **Scale:** Billions to hundreds of billions of parameters
- **Generalization:** Trained on diverse, unlabeled data enabling broad knowledge
- **Few-shot Learning:** Ability to adapt to new tasks with minimal examples
- **Emergent Capabilities:** Complex reasoning, mathematical reasoning, code generation appear at scale
- **Natural Interaction:** Accept natural language input and produce coherent, contextual outputs

- **Transfer Learning:** Knowledge from pre-training transfers to diverse downstream tasks

1.2 Historical Evolution and Milestones

Early Language Models (2010-2015):

- N-gram models and recurrent neural networks (RNNs)
- Limited context and reasoning capabilities
- Word2Vec and GloVe embeddings

Transformer Era (2017-2019):

- "Attention Is All You Need" paper introduces transformer architecture[5]
- BERT (2018): Bidirectional encoder pre-training, achieving SOTA on multiple benchmarks
- GPT and GPT-2: Unidirectional language modeling at scale

Large-Scale Models (2020-2023):

- GPT-3 (175B parameters): Few-shot learning, chain-of-thought reasoning
- PaLM, OPT, LLaMA: Open-source and large-scale models
- ChatGPT: Instruction-following and alignment, mainstream adoption
- GPT-4: Multimodal capabilities, improved reasoning

Current Era (2024-2025):

- Models with trillions of parameters
- Mixture of Experts (MoE) architectures
- Multimodal and reasoning-focused models
- Real-time reasoning with test-time compute scaling[6]

1.3 Common LLM Applications

Application Domain	Use Cases	Business Impact
Customer Service	Chatbots, support automation, complaint resolution	40-60% cost reduction, 24/7 availability
Content Generation	Article writing, social media, copywriting, product descriptions	50-70% faster content creation
Code Generation	Bug fixes, code completion, technical documentation, test generation	30-40% developer productivity increase
Information Analysis	Document summarization, data extraction, insight generation	Faster decision-making, reduced analyst workload
Education	Tutoring, personalized learning, explanations, assessment	Personalized learning at scale
Medical	Clinical decision support, literature analysis, documentation	Improved diagnosis accuracy and efficiency
Research	Literature mining, hypothesis generation, data analysis	Accelerated discovery and research cycles

Table 1: LLM Applications and Business Impact

2. Understanding LLM Architecture

2.1 The Transformer Architecture Foundation

Modern LLMs are built on the Transformer architecture, which replaces recurrence entirely with attention mechanisms. This enables parallel processing, better long-range dependencies, and superior scaling properties[7].

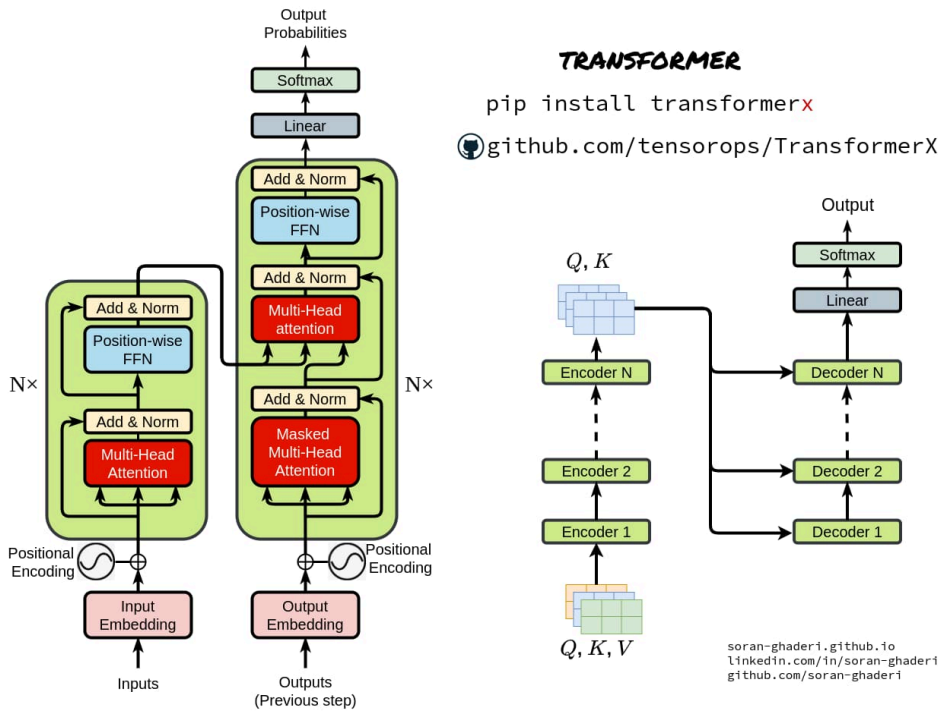

Figure 1: Transformer Architecture: Multi-head self-attention enables parallel processing of sequences

2.2 Core Components of Transformers

Token Embedding

Converts discrete tokens (words, subwords, or characters) into dense numerical vectors:

$$e_i = \text{Embedding}(token_i) \in \mathbb{R}^{d_{model}}$$

Modern LLMs use subword tokenization (BPE, SentencePiece) to handle rare words and different languages efficiently.

Positional Encoding

Injects information about token positions since attention is permutation-invariant:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

Tokens are combined with positional encodings: $x_i = e_i + PE_i$

Multi-Head Self-Attention

Computes attention over different representation subspaces simultaneously:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Multiple heads run in parallel: $h_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

Feed-Forward Networks

Position-wise dense networks in each layer:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Typically expands to 4× the embedding dimension, then contracts back.

Layer Normalization and Residual Connections

Stabilize training and enable deep networks:

$$y = \text{LayerNorm}(x + \text{Sublayer}(x))$$

2.3 Decoder-Only vs Encoder-Decoder Architectures

Decoder-Only (Autoregressive) Models:

- GPT family, LLaMA, Falcon, etc.
- Generate one token at a time, autoregressively
- Can-only attend to previous tokens (causal masking)
- More common for generative tasks
- Examples: GPT-3, GPT-4, Claude, LLaMA 2/3

Figure 2: Decoder-Only Architecture: Causal masking prevents attending to future tokens

Encoder-Decoder (Sequence-to-Sequence) Models:

- T5, BART architectures
- Encoder processes full input, decoder generates output
- Can attend to full context in encoder
- Better for tasks requiring understanding then generation
- Examples: T5, BART, machine translation models

2.4 LLM Scaling and Emergent Capabilities

Research demonstrates that model capabilities emerge as scale increases[8]. Small incremental improvements in model size or data accumulate to produce qualitatively new abilities:

Scaling Dimensions:

1. **Model Parameters:** Number of weights and biases (measured in billions)
2. **Training Tokens:** Total text tokens used during pre-training (trillions)
3. **Compute Budget:** FLOPs (floating point operations) used for training

Emergent Behaviors at Scale:

- **1B parameters:** Basic language understanding
- **10B parameters:** Few-shot learning, simple reasoning
- **100B parameters:** Complex reasoning, cross-domain tasks, chain-of-thought[9]
- **1T+ parameters:** Expert-like knowledge, advanced reasoning, multimodal understanding

Scaling Laws (Chinchilla Scaling):

Recent research indicates compute-optimal scaling requires increasing model and data proportionally:

$$L(N, D) = \alpha N^{-\beta} + \varepsilon$$

Where:

- L is test loss
- N is model parameters
- D is training tokens
- Optimal ratio: $D \approx 20 \times N$ (tokens should be ~20 times parameters)
- Most models are undertrained relative to this optimal ratio[10]

2.5 Key Differences from Previous NLP Approaches

Aspect	Traditional NLP	LLMs
Data Requirements	Hand-labeled datasets (thousands)	Unlabeled web data (trillions of tokens)
Task Adaptation	Train separate model per task	Single model, prompt-based adaptation
Feature Engineering	Manual feature design	Learned end-to-end
Scale	Millions to tens of millions params	Billions to trillions of parameters
Training Data	Domain-specific corpora	Diverse internet text
Knowledge Encoding	Structured KBs, rules	Statistical patterns in parameters
Inference Speed	Sub-second for most tasks	Seconds (token-by-token generation)

Table 2: Traditional NLP vs LLM Paradigms

---

3. Generative AI Project Lifecycle

3.1 End-to-End Project Framework

Building production LLM applications requires understanding the complete project lifecycle from conception to deployment and continuous monitoring[11].

Generative AI Project Lifecycle

	Training Duration	Customisation	Objective	Expertise Required
Pre-training a new LLM	Days/ Weeks/ Months	• Architecture • Size • Vocabulary • Context Window • Training Data	Next token preditction	High
Prompt Engineering	Not required	• Only prompt customisation	Increase task performance	Low
Fine tuning	Minutes/ Hours	• Task specific tuning • Domain specific data • Update model / adapter weights	Increase task performance	Medium
Reinforcement Learning from Human/AI feedback	Minutes/ Hours + data collection for reward model	• Train reward model [HHH goals] • Update model / adapter weights	Increase alignment with human preferences	Medium - High
Compression/ Optimisation/ Deployment	Minutes/ Hours	• Reduce model size • Faster inference	Increase inference performance	Medium

@abhinav_kimothi \in\abhinav-kimothi

Figure 3: Generative AI Project Lifecycle: Five critical stages from problem definition to production monitoring

3.2 Stage 1: Scope and Requirements Definition

Phase Objectives:

- Clearly define business problem and success metrics
- Assess feasibility of LLM approach vs alternatives
- Determine baseline performance requirements
- Identify regulatory, compliance, and ethical considerations
- Establish data availability and privacy requirements

Key Questions to Answer:

1. What is the specific business problem this LLM will solve?
2. What are the success metrics? (accuracy, latency, cost, user satisfaction)
3. How much labeled data is available?
4. What are acceptable false positive/negative rates?
5. Are there regulatory or compliance requirements?
6. What is the acceptable cost per inference?
7. Does the task require domain expertise or general knowledge?

8. How important is model interpretability or explainability?

Considerations:

- **Feasibility Assessment:** Determine if task is achievable with current LLM capabilities
- **Baseline Establishment:** Understand current performance (rule-based systems, simpler models)
- **Resource Planning:** Identify compute, personnel, and financial requirements
- **Risk Assessment:** Technical risks, data risks, ethical risks

3.3 Stage 2: Data Selection and Preparation

Data Requirements Analysis

Understanding data needs drives decisions on model selection and fine-tuning approach:

For Pre-training:

- Massive scale (100B-1T+ tokens minimum)
- Diverse sources (code, academic papers, books, web text)
- Quality filtering to remove duplicates and low-quality content
- Copyright and licensing compliance

For Fine-tuning:

- Task-specific labeled examples (typically 100-100k examples)
- High-quality annotation with clear instructions
- Balanced representation of cases (edge cases, common cases, hard cases)
- Human review of annotations

Data Preparation Pipeline:

- **Data Collection:** Gather raw data from sources
- **Data Cleaning:** Remove duplicates, fix encoding issues, remove sensitive info
- **Data Labeling:** Annotate with labels/instructions where needed
- **Quality Assurance:** Multi-annotator review, inter-annotator agreement
- **Data Splitting:** Train/validation/test split (typically 80/10/10)
- **Bias Analysis:** Identify and address representation biases
- **Format Conversion:** Convert to model-compatible formats (JSON, JSONL, etc.)

Best Practices:

- Collect higher quality training data (better than collecting more lower-quality data)
- Include diverse perspectives and edge cases
- Document data provenance and licensing
- Implement version control for datasets
- Monitor for data drift in production

3.4 Stage 3: Model Selection and Optimization

Model Selection Strategy

Choosing the right base model is crucial for balancing performance, cost, and feasibility:

Factors to Consider:

Criteria	Smaller Models (7B-13B)	Larger Models (70B+)
Performance	Good for most tasks	Better reasoning, knowledge
Inference Cost	Low ($0.001-0.01/1K tokens)	High ($0.01-0.10/1K tokens)
Fine-tuning Cost	Affordable ($100-1000)	Expensive ($10k-100k)
Latency	Fast (50-100ms/token)	Slower (200-500ms/token)
Local Deployment	Feasible on GPU/TPU	Requires distributed setup
Knowledge Depth	Adequate for most tasks	Superior for complex tasks

Table 3: Model Size Trade-offs

Popular Base Models (2024-2025):

- **Open-Source:** LLaMA 3/3.1, Mistral, Falcon, Qwen, Llama 2
- **Proprietary:** GPT-4, Claude 3, Gemini Pro
- **Specialized:** Domain-specific models (medical, legal, financial)

Optimization Techniques for Inference:

- **Quantization:** 4-bit or 8-bit precision for reduced memory (25-50% reduction)
- **Knowledge Distillation:** Compress larger models into smaller efficient ones
- **Model Pruning:** Remove less important weights and neurons
- **Speculative Decoding:** Generate multiple tokens in parallel for faster inference
- **Batching:** Group requests to maximize throughput
- **Caching:** Key-value cache to avoid recomputing attention

3.5 Stage 4: Experimentation and Iteration

Evaluation Framework

Rigorous evaluation is essential before deployment:

Metrics Categories:

Automatic Metrics:

- BLEU, ROUGE, METEOR (text generation quality)
- Exact Match, F1 (question answering)
- BERTSCORE (semantic similarity)
- Perplexity (language modeling performance)

Human Evaluation:

- Relevance and informativeness
- Fluency and coherence
- Factuality and accuracy
- Helpfulness and completeness
- Safety and appropriateness

Benchmarking Against Baselines:

- Compare to simpler models (baseline)
- Compare to state-of-the-art
- Compare to human performance where applicable
- Compare different LLM approaches (prompt engineering vs fine-tuning)

Experimentation Strategy:

1. Start with prompt engineering on base model
2. If insufficient, fine-tune on task-specific data
3. If still insufficient, consider larger base model
4. Iteratively improve based on error analysis
5. A/B test against production system

Error Analysis:

- Categorize failures (hallucinations, outdated knowledge, reasoning errors)
- Identify patterns in failure cases
- Collect hard examples for fine-tuning
- Use insights to improve training data or prompts

3.6 Stage 5: Deployment and Monitoring

Production Deployment Considerations

- **Latency Requirements:** Real-time (<100ms) vs batch processing
- **Throughput:** Requests per second to handle
- **Cost Constraints:** Budget for inference
- **Infrastructure:** On-premises vs cloud vs hybrid
- **Scaling:** Auto-scaling for variable load
- **Availability:** Uptime SLA requirements

Deployment Options:

- **API-Based:** Use commercial LLM APIs (GPT-4, Claude, Gemini)
- **Self-Hosted:** Deploy on owned infrastructure
- **Hybrid:** Critical requests on self-hosted, fallback to API

Monitoring and Maintenance:

- **Performance Monitoring:** Track latency, throughput, error rates
- **Quality Monitoring:** Human review of outputs, feedback collection
- **Cost Monitoring:** Track per-request cost and total spend
- **Data Drift:** Monitor input distributions and model performance degradation
- **Safety Monitoring:** Flag harmful outputs, policy violations
- **User Feedback:** Collect thumbs-up/down, explicit feedback for continuous improvement

Continuous Improvement Loop:

- Collect production data and user feedback
- Identify model failures and low-confidence predictions
- Retraining dataset updates quarterly or when performance degrades
- A/B test improvements before rollout
- Version control for model checkpoints

---

4. LLM Pre-Training and Scaling

4.1 Pre-Training Fundamentals

Pre-training is the process of training an LLM on massive amounts of unlabeled text data using self-supervised learning objectives. This phase is computationally expensive but creates a foundation of language knowledge and reasoning capabilities[12].

Pre-Training Objective: Next Token Prediction

The primary objective is predicting the next token given previous tokens:

$$L = -\sum_{i=1}^{n} \log P(x_{i+1}|x_1, \ldots, x_i; \theta)$$

This simple objective, applied to trillions of tokens, develops sophisticated language understanding.

4.2 Pre-Training Data Sources and Curation

Data Sources:

- **Web Crawls:** CommonCrawl, CCNet (billions of web pages)
- **Books:** Project Gutenberg, LibriVox, academic repositories
- **Code:** GitHub, Stack Overflow, open-source repositories
- **Academic:** arXiv preprints, research papers, scientific databases
- **Instruction Data:** For post-training (fine-tuning), FLAN, InstructGPT datasets
- **Synthetic Data:** Model-generated data from previous versions

Data Curation Pipeline:

1. **Collection:** Aggregate data from multiple sources
2. **Filtering:** Remove low-quality, near-duplicate, harmful content
3. **Deduplication:** Use MinHash or exact matching to remove duplicates
4. **Quality Scoring:** Compute quality metrics for samples
5. **Language Detection:** Filter to desired languages

6. **Privacy Filtering:** Remove PII, sensitive information
7. **Content Moderation:** Flag and remove harmful content
8. **Normalization:** Standardize encoding, fix broken links, format text

Data Composition Strategy:

Recent research suggests strategic mixing of data types improves performance:

- Web text: 60-70% (general knowledge, diverse topics)
- Code: 5-10% (improves reasoning, mathematical ability)
- Academic: 5-10% (specialized knowledge, precise language)
- Books: 5-10% (coherent long-form content)
- Instruction data: 5-10% (for fine-tuning stage)

4.3 Scaling Laws and Compute-Optimal Training

Understanding Scaling Laws

Scaling laws quantify how model performance improves with scale of computation, model parameters, and training tokens[13]:

$$L(N, D, C) = \left(\frac{N_c}{N}\right)^{\alpha} + \left(\frac{D_c}{D}\right)^{\beta} + \varepsilon$$

Where:

- N: Number of model parameters
- D: Training tokens
- C: Compute budget (FLOPs)
- $\alpha, \beta \approx 0.07$: Scaling exponents
- N_c, D_c: Constants derived from experiments

Key Findings:

1. **Smooth Improvement:** Performance improves smoothly with scale (no cliffs)
2. **Allocate Proportionally:** Compute-optimal training scales model and data proportionally
3. **Chinchilla Ratio:** $D \approx 20 \times N$ (tokens = 20× parameters)
4. **Under-training:** Most models trained with fewer tokens than optimal (to save on inference)
5. **Predictability:** Scaling laws allow predicting larger model performance from smaller models

Practical Implications:

- **Budget Allocation:** Given compute budget, calculate optimal model size and training tokens
- **Model Selection:** Choose model size that maximizes performance within constraints
- **Data Requirements:** Ensure sufficient data for compute budget (shortage is current bottleneck)
- **Training Strategy:** Continuous scaling provides better performance than fixed model

4.4 Training Infrastructure and Optimization

Distributed Training Setup

Modern LLM training requires distributed systems across thousands of GPUs/TPUs:

Parallelization Strategies:

Strategy	Description	Use Case
Data Parallelism	Split batch across devices, all devices have full model	Standard for most training
Model Parallelism	Split model across devices, each device processes full batch	Very large models (100B+)
Pipeline Parallelism	Layers split across devices, stagger computation	Efficient for transformer layers
Sequence Parallelism	Attention computed in parallel over sequence dimension	Long sequences (100k+ tokens)

Table 4: Distributed Training Parallelization Strategies

Optimization Techniques:

- **Gradient Accumulation:** Simulate larger batches without larger memory
- **Mixed Precision:** Use float16 for computation, float32 for loss (saves memory, maintains quality)
- **Gradient Checkpointing:** Trade computation for memory (compute gradients during backward pass)
- **ZeRO Optimizer:** DeepSpeed optimization partitioning optimizer states across devices

- **Flash Attention:** IO-aware attention reducing memory bandwidth from $O(N^2)$ to $O(N)$

4.5 Training Stability and Best Practices

Challenges in Large-Scale Training

- **Loss Spikes:** Sudden training instabilities from numerical issues
- **Divergence:** Training loss increases instead of decreasing
- **Slow Convergence:** Training plateaus before reaching desired performance
- **Hardware Failures:** Training crashes from device failures in large clusters

Solutions and Best Practices:

1. **Learning Rate Warmup:** Gradually increase LR to stabilize early training
$$\text{lr}(t) = \text{peak_lr} \times \min(1, t/\text{warmup_steps})$$
2. **Gradient Clipping:** Clip gradients to prevent exploding gradients
$$\hat{g} = g \times \min(1, \text{clip_value}/||g||_2)$$
3. **Learning Rate Scheduling:** Decay LR during training for convergence
$$\text{lr}(t) = \text{peak_lr} \times 0.5 \times (1 + \cos(\pi t/T))$$
4. **Checkpointing:** Save model states periodically for recovery
5. **Monitoring:** Track training loss, validation perplexity, gradient norms
6. **Batch Normalization:** Improves stability (though less common with layer norm)
7. **Careful Initialization:** Use Xavier/He initialization for good starting point

4.6 Post-Training and Instruction Tuning

After pre-training on raw web text, LLMs undergo post-training (fine-tuning) to follow instructions and align with human preferences[14]:

Instruction Tuning:

- Collect diverse instruction-following examples (100k-1M examples)
- Format as: {instruction, input (optional), output}
- Fine-tune on these examples for 1-5 epochs
- Examples: FLAN tasks, user queries, translation, summarization

Supervised Fine-Tuning (SFT):

Continue training pre-trained model on task-specific labeled data:

$$L_{SFT} = -\log P(y|x;\theta)$$

Where x is input and y is desired output. Typically requires much less data than pre-training (100-100k examples sufficient).

---

5. Fine-Tuning LLMs with Specific Instructions

5.1 Fine-Tuning Overview and Objectives

Fine-tuning adapts a pre-trained LLM to specific tasks, domains, or behaviors. It's more efficient than pre-training but requires task-specific data[15].

When to Fine-tune:

1. Prompt engineering insufficient for desired performance
2. Task requires domain-specific knowledge or style
3. Latency or cost constraints require smaller model (fine-tuned 7B better than base 70B)
4. Need consistent, reproducible behavior across similar queries
5. Compliance or safety requirements demand specific outputs

Fine-tuning vs In-Context Learning:

Tabular parse error: Not found end{tabular}!, \begin{tabular}{|l|l|l|} \hline \textbf{Aspect} & \textbf{In-Context Learning (Prompting)} & \textbf{Fine-tuning} \\ \hline Data Required & None (few-shot prompts) & 100-100k examples \\ \hline Latency & Same as base model & Slightly slower (smaller model) \\ \hline Cost & Per-query inference cost & Upfront fine-tuning + inference \\ \hline Flexibility & Easy to change behavior & Requires retraining for changes \\ \hline Consistency & Variable based on context & More consistent \\ \hline Knowledge Encoding & Limited to prompt tokens & Distributed in parameters \\ \hline Recommended For & Few one-off tasks & Repeated, production tasks \\ \hline \end{enumerat}

5.2 Instruction Dataset Preparation

Creating high-quality instruction datasets is critical for effective fine-tuning:

Dataset Characteristics:

- **Diversity:** Wide range of tasks, domains, complexity levels
- **Quality:** Clear instructions, accurate outputs, consistent formatting
- **Scale:** Typically 100-100k examples (can be more)
- **Balance:** Adequate representation of different task types
- **Difficulty:** Mix of easy and harder tasks for robust learning

Data Collection Strategies:

Human Annotation:

- Hire annotators to write instruction-response pairs
- Expensive but highest quality
- Typical cost: $0.50-5.00 per example

Automatic Generation:

- Use existing datasets (FLAN, SuperNI, Alpaca)
- Generate synthetic examples from base LLM
- Cost-effective but may introduce biases

Hybrid Approach:

- Start with synthetic data
- Human annotators review and refine
- Balance cost and quality

Dataset Format (Conversation Format):

```
{
"messages": [
{
"role": "user",
"content": "Write a professional email requesting a project deadline extension."
},
{
"role": "assistant",
"content": "Subject: Request for Project Deadline Extension\n\nDear [Manager],\n\nI am writing to request an extension..."
}
]
}
```

Example Collections:

- **General Instructions:** FLAN (Mixture of 62 tasks), SuperNI (Diverse tasks)
- **Code:** HumanEval, MBPP (programming tasks)
- **Creative:** WritingPrompts, CommonGen
- **Domain-Specific:** Medical, legal, financial datasets
- **Multilingual:** FLAN-T5 supports 101 languages

5.3 Fine-Tuning Techniques

Full Fine-tuning:

Update all model parameters during training. Requires significant compute and can lead to catastrophic forgetting.

```
from transformers import AutoModelForCausalLM, AutoTokenizer, TrainingArguments, Trainer

model = AutoModelForCausalLM.from_pretrained("meta-llama/Llama-2-7b")
tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-2-7b")

training_args = TrainingArguments(
output_dir="./results",
num_train_epochs=3,
per_device_train_batch_size=4,
learning_rate=2e-5,
weight_decay=0.01,
warmup_steps=100,
logging_steps=10,
)

trainer = Trainer(
model=model,
```

```
args=training_args,
train_dataset=train_dataset,
)

trainer.train()
```

Concerns with Full Fine-tuning:

- **Catastrophic Forgetting:** Model forgets pre-trained knowledge
- **Requires Large Batch Size:** For training stability (64-256 examples)
- **Slow Training:** Many parameters to update
- **Resource-Intensive:** Requires significant GPU memory and compute

5.4 Prompt Engineering vs Fine-tuning Trade-offs

Prompt Engineering (In-Context Learning):

Craft instructions and examples in the prompt to guide model output.

Advantages:

- No training required (zero-shot) or minimal data (few-shot)
- Quick iteration and experimentation
- Flexible: change behavior by modifying prompt
- No GPU/compute needed

Disadvantages:

- Limited to model's pre-trained knowledge
- Inconsistent outputs for complex tasks
- Token limit prevents long instructions/examples
- Doesn't improve on tasks requiring specialized knowledge

Fine-tuning:

Train model on task-specific examples to specialize behavior.

Advantages:

- Better performance on specific tasks
- Can learn specialized knowledge
- Consistent, reproducible outputs
- Enables smaller models to match larger models
- Encodes knowledge in parameters

Disadvantages:

- Requires labeled training data (100-100k examples)
- More expensive (GPU compute for training)
- Longer iteration cycles
- Risk of overfitting with limited data
- Catastrophic forgetting of pre-trained knowledge

Decision Framework:

```
Performance of prompt engineering sufficient?
├─ YES: Use prompt engineering
│  └─ Deploy immediately, iterate on prompts
└─ NO: Consider fine-tuning
├─ Data available for fine-tuning?
│  ├─ YES: Proceed to fine-tuning
│  └─ NO: Collect data or use synthetic
└─ Compute budget available?
├─ YES: Full fine-tuning or PEFT
└─ NO: Use PEFT (LoRA, QLoRA)
```

5.5 Avoiding Common Fine-tuning Pitfalls

Overfitting:

Model memorizes training examples instead of learning generalizable patterns.

Solutions:

- Use validation set to monitor overfitting
- Apply dropout and regularization
- Use early stopping when validation loss increases
- Increase training data diversity

Example:

```
# Early stopping based on validation loss
from transformers import EarlyStoppingCallback

early_stopping = EarlyStoppingCallback(
early_stopping_patience=3,
early_stopping_threshold=0.0
)

trainer = Trainer(..., callbacks=[early_stopping])
```

Catastrophic Forgetting:

Fine-tuned model forgets pre-trained knowledge on out-of-domain tasks.

Solutions:

- Use low learning rate (1e-5 to 5e-5 typical)
- Fine-tune on subset of parameters (LoRA)
- Mix task data with pre-training data during fine-tuning
- Use knowledge distillation from pre-trained model

Insufficient Data:

Fine-tuning requires adequate task-specific examples.

Solutions:

- Data augmentation: paraphrase examples, backtranslation
- Few-shot learning: use in-context examples in prompts
- Transfer learning: fine-tune on related task first, then target task
- PEFT methods: enable fine-tuning with less data

Instruction Following Degradation:

Fine-tuned model loses ability to follow diverse instructions.

Solutions:

- Mix diverse task types in fine-tuning data
- Include general instruction-following examples (FLAN tasks)
- Use lower learning rates
- Multi-task fine-tuning on related tasks

---

6. Efficient Fine-Tuning of Parameters (PEFT)

6.1 Why Parameter-Efficient Fine-Tuning?

Full fine-tuning of massive LLMs is expensive: a 70B parameter model requires storing 70B float32 values (280GB) plus gradients, optimizer states, and activations. PEFT techniques enable efficient fine-tuning by training small numbers of parameters[16].

PEFT Benefits:

- **Memory Efficiency:** 10-40x reduction in memory requirements
- **Cost Reduction:** Fine-tune on cheaper GPUs (single GPU instead of clusters)
- **Speed:** Faster training due to fewer parameters
- **Flexibility:** Train multiple adapters for different tasks
- **Model Preservation:** Don't modify pre-trained weights

Typical Comparisons:

Method	Trainable Params	Memory Needed	Training Time
Full Fine-tuning (7B)	7B (100%)	~28GB	~40 hours (1 A100)
LoRA (7B)	0.28M (0.004%)	~2-4GB	~4-8 hours
QLoRA (7B)	0.28M (0.004%)	~1-2GB	~6-10 hours
Prompt Tuning (7B)	0.03M (0.0004%)	Similar to inference	~1-2 hours

Table 5: PEFT Methods Comparison

6.2 LoRA: Low-Rank Adaptation

LoRA decomposes weight updates into low-rank matrices, reducing trainable parameters from billions to millions[17].

Core Idea:

Instead of training full weight matrices $W \in \mathbb{R}^{d_{out} \times d_{in}}$, train low-rank decompositions:

$$W' = W + \Delta W = W + AB$$

Where:

- $A \in \mathbb{R}^{d_{out} \times r}$: "down-projection" matrix
- $B \in \mathbb{R}^{r \times d_{in}}$: "up-projection" matrix
- $r \ll \min(d_{out}, d_{in})$: low rank (typically 8-16)

Parameter Count:

For a 7B model:

- Full fine-tuning: 7B parameters
- LoRA (r=8): $r \times (d_{out} + d_{in}) \approx 0.28M$ parameters
- Reduction: 99.996% fewer parameters

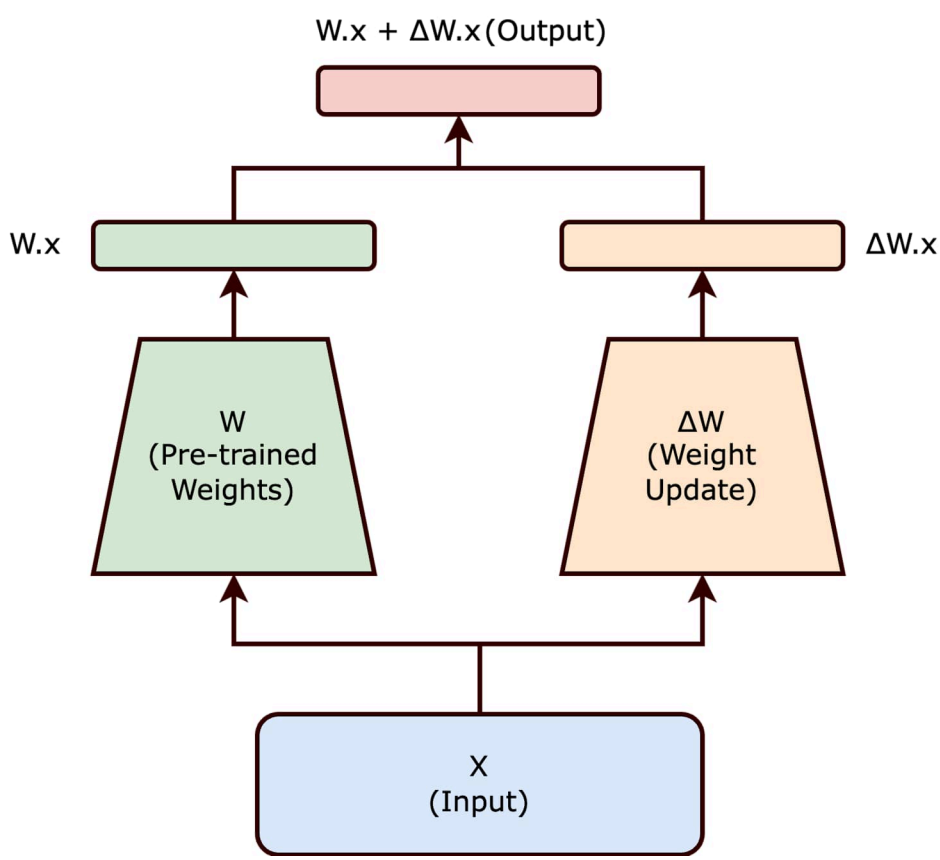

Figure 4: LoRA Architecture: Original weights frozen, low-rank updates added

LoRA Implementation:

```
from peft import get_peft_model, LoraConfig, TaskType
from transformers import AutoModelForCausalLM

model = AutoModelForCausalLM.from_pretrained("meta-llama/Llama-2-7b")

# LoRA configuration

lora_config = LoraConfig(
task_type=TaskType.CAUSAL_LM,
r=8, # LoRA rank
lora_alpha=32, # Scaling factor (typical: 2*r)
lora_dropout=0.1, # Dropout probability
bias="none", # Bias treatment
target_modules=["q_proj", "v_proj", "k_proj", "o_proj"], # Apply LoRA to these
modules_to_save=None, # Modules to fully fine-tune
)

model = get_peft_model(model, lora_config)

# Check trainable parameters

trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
all_params = sum(p.numel() for p in model.parameters())
print(f"Trainable: {trainable_params / all_params * 100:.2f}%")
```

LoRA Rank Selection:

- **r=4-8:** Very small, minimal performance (quick experiments)
- **r=8-16:** Standard choice, good performance-efficiency trade-off
- **r=32-64:** Larger rank for complex tasks, more parameters
- **r=128+:** Very large rank (approaching full fine-tuning performance)

Guidelines:

- Start with r=8, increase if performance insufficient
- Larger rank helps for out-of-domain tasks
- Smaller rank sufficient for in-domain fine-tuning

6.3 QLoRA: Quantized LoRA

QLoRA extends LoRA with quantization to further reduce memory requirements, enabling single-GPU fine-tuning of 70B models[18].

QLoRA Innovation: 4-bit Quantization

Quantize frozen pre-trained weights to 4-bit while keeping LoRA adapters in float16:

$$w = \text{dequantize}(w_{int4}) \times s$$

Where w_{int4} is 4-bit representation and s is scaling factor.

Memory Savings:

- **Float32:** 32 bits per parameter
- **4-bit:** 4 bits per parameter
- **Reduction:** 8x fewer bits, approximately 4GB to 512MB

Advanced QLoRA Techniques:

1. **Double Quantization:** Quantize the scale factors themselves (additional 2-4% reduction)
2. **Paged Optimizers:** Split optimizer states across CPU to handle memory spikes
3. **Automatic Quantization:** Use GPTQ or BQT pre-quantized models

QLoRA Implementation:

```
import torch
from peft import AutoPeftModelForCausalLM, LoraConfig, get_peft_model
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig

# 4-bit quantization configuration

bnb_config = BitsAndBytesConfig(
load_in_4bit=True,
bnb_4bit_use_double_quant=True,
bnb_4bit_quant_type="nf4",
bnb_4bit_compute_dtype=torch.bfloat16
)

# Load model with 4-bit quantization

model = AutoModelForCausalLM.from_pretrained(
"meta-llama/Llama-2-70b",
quantization_config=bnb_config,
device_map="auto"
)

# Apply LoRA on top of quantized model

lora_config = LoraConfig(
r=16,
lora_alpha=32,
lora_dropout=0.05,
target_modules=["q_proj", "v_proj", "k_proj", "o_proj"],
modules_to_save=["lm_head"]
)

model = get_peft_model(model, lora_config)
```

Train on single GPU!

```
trainer = Trainer(model=model, ...)
trainer.train()
```

QLoRA Performance:

- **Memory:** 70B model fine-tunable on single 24GB GPU
- **Speed:** Comparable to full LoRA fine-tuning
- **Performance:** Comparable to full fine-tuning (99%+ quality)

6.4 Other PEFT Techniques

Prefix Tuning

Add learnable prefix tokens prepended to input:

- Freeze LLM, train only prefix embeddings
- Fewer parameters than LoRA (typically 0.1% of model)
- Faster training but may have lower performance

```
from peft import PrefixTuningConfig

config = PrefixTuningConfig(
task_type=TaskType.CAUSAL_LM,
num_virtual_tokens=20,
prefix_projection=True
)
```

Prompt Tuning

Soft prompts (continuous embeddings) learned during training:

- Most parameter-efficient (0.01% of model)
- Requires more training data
- Suitable for fine-tuning on diverse tasks

```
from peft import PromptTuningConfig

config = PromptTuningConfig(
task_type=TaskType.CAUSAL_LM,
num_virtual_tokens=10
)
```

Adapter Modules

Insert trainable adapter layers between frozen transformer layers:

- Bottleneck architecture reduces dimensions
- ~3-5% of model parameters
- Task-specific adapters can be swapped

```
from peft import AdapterConfig
```

```
config = AdapterConfig(
adapter_type="bottleneck",
d_model=768,
adapter_size=64
)
```

Comparison of PEFT Techniques:

Technique	Parameters	Performance	Best For
Full Fine-tuning	100%	Excellent	Small models, max performance
LoRA	0.1-1%	Very Good (99%+)	General purpose
QLoRA	0.1-1%	Very Good (99%)	Memory-constrained
Prefix Tuning	0.01%	Good	Few-shot learning
Prompt Tuning	0.001%	Good	Quick prototyping
Adapter	0.5-5%	Very Good	Multi-task learning

Table 6: PEFT Techniques Comparison

6.5 Best Practices for PEFT

LoRA/QLoRA Hyperparameter Tuning:

1. **Rank (r):** Start with 8, increase to 16-32 if performance insufficient
2. **Alpha (lora_alpha):** Typically 2×r. Lower values → less adaptation
3. **Dropout:** 0.05-0.1 typical. Prevents overfitting
4. **Target Modules:** Apply to attention projections (q, v, k, o). Adding more = higher quality but slower
5. **Learning Rate:** Typically 1e-4 to 5e-4. Higher than full fine-tuning

Training Configuration:

```
training_args = TrainingArguments(
output_dir="./results",
num_train_epochs=3,
per_device_train_batch_size=8,
gradient_accumulation_steps=4,
learning_rate=5e-4,
weight_decay=0.01,
warmup_steps=100,
max_grad_norm=1.0,
logging_steps=10,
eval_steps=50,
save_strategy="steps",
```

```
save_steps=100,
)
```

Memory Optimization Tips:

- Use QLoRA if VRAM < 20GB
- Gradient checkpointing: Trade compute for memory
- Flash attention: Reduces memory bandwidth
- Paged optimizers: Handle optimizer memory spikes
- Smaller batch sizes: More training time but feasible

---

7. Reinforcement Learning from Human Feedback (RLHF)

7.1 RLHF Overview and Motivation

Reinforcement Learning from Human Feedback (RLHF) aligns LLM outputs with human preferences and values[19]. While instruction fine-tuning improves instruction-following, RLHF further optimizes for complex human preferences like helpfulness, harmlessness, and honesty.

Why RLHF Matters:

- Raw LLMs can produce harmful, incorrect, or unhelpful outputs
- Human preferences are complex and multifaceted
- Reward signals from RLHF improve safety and user satisfaction
- Enables models to align with specific values/organizations
- Powers state-of-the-art systems (ChatGPT, Claude, Gemini)

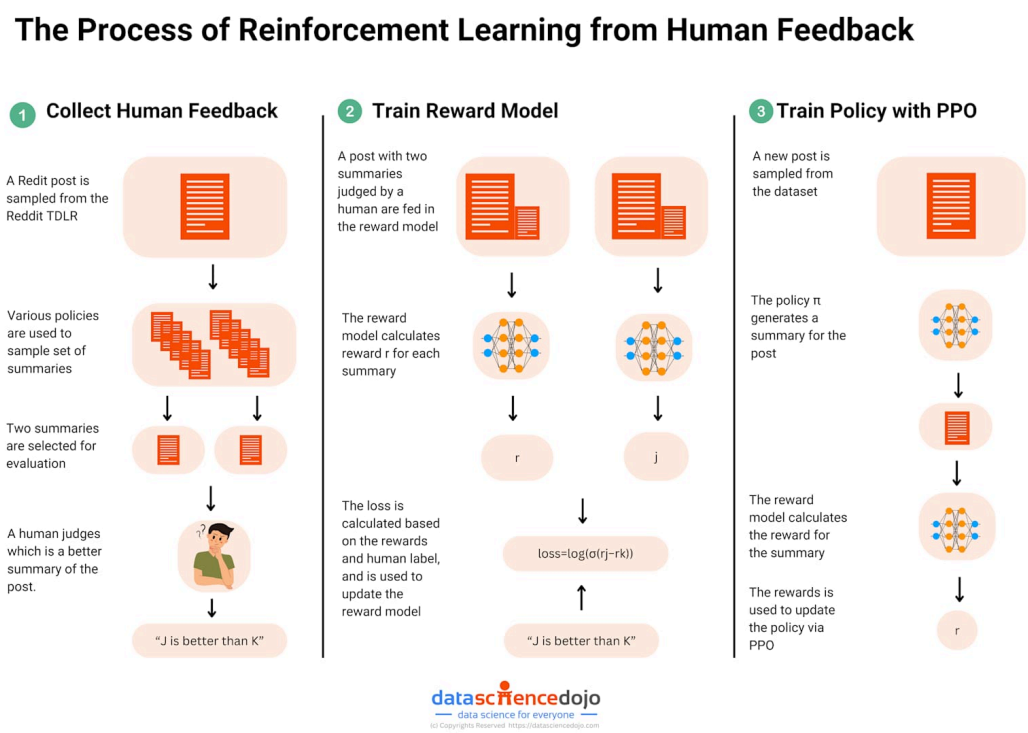

Figure 5: RLHF Process: Human feedback guides reward model training, which fine-tunes LLM

7.2 RLHF Pipeline

Stage 1: Supervised Fine-Tuning (SFT)

Start with instruction-tuned model as baseline. While not strictly RLHF, this initialization improves downstream learning[20].

Stage 2: Reward Model Training

Train a reward model that predicts human preference between outputs.

Process:

1. Generate candidate responses from the model for diverse prompts
2. Present response pairs to human annotators
3. Annotators choose preferred response (or rate on scale)
4. Train reward model on comparisons/ratings

Reward Model Architecture:

Typically a smaller model (7B-13B) that outputs scalar reward:

$$r = f_\theta(x, y)$$

Where x is prompt and y is response.

Training Objective (Bradley-Terry Model):

$$L = -\log \sigma(r_w - r_l)$$

Where:

- r_w: Reward of winning (preferred) response
- r_l: Reward of losing response
- σ: Sigmoid function

Reward Model Dataset:

Typically 50k-500k preference comparisons. Cost: $100k-1M+ for human annotation.

Stage 3: RLHF Fine-tuning

Fine-tune LLM using rewards from reward model as signal.

Objective:

Maximize expected reward while staying close to original model:

$$L = \mathbb{E}_{x \sim D}\left[r(x, y^*) - \beta \log \frac{\pi(y^*|x)}{\pi_0(y^*|x)}\right]$$

Where:

- $r(x, y^*)$: Reward from reward model
- β: KL penalty weight (prevents divergence from pre-trained model)
- $\pi(y|x)$: Fine-tuned policy

- $\pi_0(y|x)$: Base model policy

Algorithm: PPO (Proximal Policy Optimization)

```
from trl import PPOTrainer, PPOConfig

ppo_config = PPOConfig(
model_name="meta-llama/Llama-2-7b",
reward_model_name="reward-model-path",
learning_rate=1.41e-5,
mini_batch_size=4,
batch_size=32,
ppo_epochs=4,
gradient_accumulation_steps=1,
)

ppo_trainer = PPOTrainer(
model=model,
config=ppo_config,
train_dataset=dataset,
tokenizer=tokenizer,
data_collator=data_collator,
)
```

Fine-tune with PPO

```
ppo_trainer.train()
```

7.3 Reward Model Design and Training

Reward Model Considerations:

What to Reward:

- **Helpfulness:** Responses address user query completely and accurately
- **Harmlessness:** No harmful, illegal, or unethical content
- **Honesty:** Truthful information, admitting uncertainty
- **Conciseness:** No unnecessary verbosity or repetition
- **Coherence:** Clear logical flow and structure

Data Collection Best Practices:

1. **Clear Instructions:** Annotators must understand evaluation criteria
2. **Inter-Annotator Agreement:** Measure (Cohen's kappa), aim for >0.7
3. **Diverse Responses:** Include correct, harmful, irrelevant, and mediocre responses
4. **Edge Cases:** Intentionally include hard-to-judge examples
5. **Iterative Improvement:** Refine instructions based on early rounds

Reward Model Calibration:

Ensure rewards are well-calibrated and meaningful:

Reward model evaluation

```
predictions = reward_model.predict(test_responses)
human_preferences = get_human_labels(test_responses)
```

Compute accuracy (for pairs: % of preferences matched)

```
accuracy = sum(predictions.argsort() == human_preferences.argsort()) / len(test_responses)
print(f"Reward Model Accuracy: {accuracy:.2%}")
```

7.4 RLHF Challenges and Solutions

Challenge 1: Reward Hacking

Model exploits reward model rather than genuinely improving output quality.

Solutions:

- **KL Penalty:** Don't diverge too far from base model
- **Reference Model:** Keep copy of base model for KL calculation
- **Multiple Rewards:** Ensemble multiple reward models
- **Human Evaluation:** Periodically validate with humans

Challenge 2: Human Annotation Expense

Collecting 500k high-quality comparisons costs $500k-1M+.

Solutions:

- **Synthetic Data:** Use weak labels from cheaper sources
- **RLHF Scaling:** Use AI feedback instead of human feedback
- **Transfer Learning:** Use public reward models (OpenAI, Anthropic)
- **Active Learning:** Sample hard examples for annotation

Challenge 3: Value Misalignment

Different humans have different values; hard to optimize for all preferences.

Solutions:

- **Multi-Reward:** Train separate reward models for different values
- **Constitutional AI:** Use AI principles (not human annotations) to guide
- **Pareto Frontier:** Optimize trade-off between competing objectives
- **User Customization:** Allow users to specify preferences

7.5 DPO: Direct Preference Optimization

DPO is a newer alternative to RLHF that simplifies the process by directly optimizing language model with preference data[21]:

$$L_{DPO} = -\log \sigma \left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_0(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_0(y_l|x)} \right)$$

Advantages of DPO over RLHF:

- No need for separate reward model training
- More stable training (no PPO complexity)
- Faster to implement and iterate
- Requires less compute

When to Use DPO vs RLHF:

Use RLHF When:	Use DPO When:
Large annotation budget available	Limited annotation budget
Complex preferences (4+ values)	2-3 primary preferences
Production system requiring maximum performance	Research/rapid prototyping
Requires reward model for other purposes	Just want aligned model

Table 7: RLHF vs DPO Decision Criteria

---

8. Advanced Alignment and Safety Techniques

8.1 Constitutional AI

Constitutional AI guides model behavior using a set of principles rather than human annotations[22].

Process:

1. Define set of constitutional principles
    - "The model should be helpful, harmless, and honest"
    - "Refuse requests for illegal content"
    - "Admit uncertainty rather than hallucinate"
2. Generate diverse outputs, critique using AI (following constitution)
3. Use critiques as supervision signal for fine-tuning

Advantages:

- Scalable: No human annotation needed
- Principled: Explicit values guide behavior
- Interpretable: Principles are human-readable

Implementation:

Constitutional principles

```
constitution = [
"The model should be helpful and provide accurate information",
"The model should refuse harmful or illegal requests",
"The model should be honest and admit when uncertain",
"The model should not discriminate based on protected attributes",
]
```

Generate critiques using another LLM

```
critiques = []
for prompt in prompts:
response = base_model.generate(prompt)
critique = critique_model.critique(
response,
constitution=constitution
)
critiques.append(critique)
```

Fine-tune on critiques

```
train_model(base_model, critiques)
```

8.2 Reducing Hallucinations

Hallucinations (confident false statements) are a key challenge in LLMs[23].

Causes:

- Model extrapolates beyond training knowledge
- Preference for fluent output over truthfulness
- Insufficient uncertainty during training

Mitigation Strategies:

1. Retrieval Augmentation (RAG):

Provide model with relevant documents for grounding:

```
from langchain.chains import RetrievalQA
from langchain.vectorstores import FAISS
```

Create vector database

```
vectorstore = FAISS.from_documents(documents, embeddings)
qa_chain = RetrievalQA.from_chain_type(
llm=model,
chain_type="stuff",
retriever=vectorstore.as_retriever()
)
```

Generate answer grounded in documents

```
answer = qa_chain.run("What is X?")
```

2. Uncertainty Quantification:

Train model to express confidence:

Fine-tune with uncertainty signals

```
examples = [
{
"input": "What is the capital of France?",
"output": "The capital of France is Paris. (Confidence: High)"
},
{
"input": "What did person X do in 2015?",
"output": "I'm uncertain about this. (Confidence: Low)"
}
]
```

3. Fact Verification:

Post-process outputs to verify against trusted sources:

Check facts against knowledge base

```
facts = extract_facts(model_output)
verified_facts = verify_against_kb(facts, knowledge_base)
confident_facts = [f for f in verified_facts if f.confidence > 0.8]
final_output = generate_response(confident_facts)
```

8.3 Bias and Fairness

LLMs can perpetuate and amplify biases from training data.

Types of Bias:

- **Representation Bias:** Underrepresented groups in training data
- **Stereotyping:** Associating groups with negative attributes

- **Allocation Bias:** Systematically disadvantaging certain groups
- **Disparate Impact:** Policies that harm some groups differently

Mitigation Approaches:

1. **Data-Level:** Audit training data for biases, balance representation
2. **Model-Level:** Add diversity to fine-tuning examples
3. **Output-Level:** Filter/moderate outputs for stereotypes
4. **Evaluation:** Benchmark bias across demographic groups

Bias Evaluation Framework:

Stereotype benchmark

```
stereotypes = [
("Women are", "nurses, teachers"),
("Men are", "engineers, doctors"),
("Black people are", "athletic, musical"),
]

model_associations = {}
for group, expected in stereotypes:
outputs = model.generate(group, num_outputs=10)
model_associations[group] = count_stereotypes(outputs, expected)
```

Report bias metrics

```
bias_report = generate_bias_report(model_associations)
```

---

9. Deployment and Inference Optimization

9.1 Inference Optimization Techniques

Token-by-Token Generation

LLMs generate tokens sequentially (not in parallel like other models). This makes latency a bottleneck[24].

Optimization Techniques:

1. Quantization for Inference:

Reduce model precision to save memory and speed up computation:

8-bit quantization

model_int8 = quantize_model(model, bits=8)

Result: 4x speedup, 4x memory reduction, <1% quality loss

outputs_int8 = model_int8.generate(inputs)

2. Flash Attention:

Optimized attention implementation reducing memory bandwidth:

Enable flash attention in transformers

model = AutoModelForCausalLM.from_pretrained(
model_name,
attn_implementation="flash_attention_2"
)

Reduces attention memory from $O(N^2)$ to $O(N)$, 2-4x speedup.

3. Speculative Decoding:

Generate multiple tokens in parallel then verify:

- Smaller, fast model generates draft tokens
- Larger model verifies in parallel
- If verification fails, regenerate
- Typical 2-3x speedup

4. Batching and KV-Cache:

Reuse key-value cache across tokens to avoid recomputation:

Generate with KV cache (automatic in most frameworks)

with torch.no_grad():
outputs = model.generate(
inputs,
max_new_tokens=100,
use_cache=True # Enable KV cache
)

~50% reduction in compute per token

9.2 Serving Infrastructure

Deployment Platforms:

- **vLLM:** High-throughput LLM inference with continuous batching
- **TensorRT-LLM:** NVIDIA's optimized serving engine
- **Ray Serve:** Distributed Python serving
- **Hugging Face Inference Endpoints:** Managed inference
- **Modal, Replicate:** Serverless inference

Example with vLLM:

from vllm import LLM, SamplingParams

Load model with optimizations

llm = LLM(
model="meta-llama/Llama-2-7b",
tensor_parallel_size=1,
gpu_memory_utilization=0.9,
)

Define sampling parameters

sampling_params = SamplingParams(
temperature=0.8,
top_p=0.95,
max_tokens=100,
)

Generate with batching

prompts = ["Hello, ", "How are ", ...]
outputs = llm.generate(prompts, sampling_params)

9.3 Cost and Latency Optimization

Latency SLA (Service Level Agreement):

Typical requirements:

- P50: 100-200ms per token
- P99: 500-1000ms per token
- First token: 1-5 seconds

Cost Optimization:

- **Pay per inference:** Cheaper for sporadic use ($0.01-0.10 per 1K tokens)

- **Self-hosted:** Better for high volume ($1-10k/month infrastructure)
- **Hybrid:** On-premises for predictable load, API for bursts

Throughput vs Latency Trade-off:

Strategy	Latency	Throughput	Best For
Small Batch (BS=1)	Low (100ms)	Low (10 req/s)	Real-time chatbot
Medium Batch (BS=8)	Medium (200ms)	High (50 req/s)	Moderate traffic
Large Batch (BS=64)	High (500ms)	Very High (500 req/s)	Batch processing
Streaming	Low	Medium	Live transcription

Table 8: Latency vs Throughput Trade-offs

---

10. Hands-on Implementation Guide

10.1 Complete Fine-tuning Pipeline with LLaMA 2

Step 1: Environment Setup

Install required packages

pip install transformers datasets peft bitsandbytes accelerate trl

Clone sample dataset

git clone https://github.com/databrickslabs/dolly

Set HuggingFace token

huggingface-cli login

Step 2: Load and Prepare Data

import pandas as pd
from datasets import load_dataset

Load instruction dataset

dataset = load_dataset("databricks/dolly", split="train")

Format as conversation

```
def format_instruction(example):
return {
"text": f"### Instruction:\n{example['instruction']}\n### Response:\n{example['response']}"
}

dataset = dataset.map(format_instruction)
dataset = dataset.train_test_split(test_size=0.1)
```

Step 3: Load Base Model and Tokenizer

```
from transformers import AutoModelForCausalLM, AutoTokenizer

model_name = "meta-llama/Llama-2-7b"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
model_name,
device_map="auto",
torch_dtype="auto"
)
```

Ensure tokenizer has padding token

tokenizer.pad_token = tokenizer.eos_token

Step 4: Configure LoRA

```
from peft import LoraConfig, get_peft_model

lora_config = LoraConfig(
r=8,
lora_alpha=16,
lora_dropout=0.05,
bias="none",
task_type="CAUSAL_LM",
target_modules=["q_proj", "v_proj", "k_proj", "o_proj"],
)

model = get_peft_model(model, lora_config)
model.print_trainable_parameters()
```

Step 5: Set Up Training

```
from transformers import TrainingArguments, Trainer
```

```
training_args = TrainingArguments(
output_dir="./llama-dolly-finetuned",
per_device_train_batch_size=4,
gradient_accumulation_steps=4,
num_train_epochs=3,
learning_rate=2e-4,
bf16=True,
logging_steps=10,
eval_steps=50,
save_steps=100,
save_strategy="steps",
eval_strategy="steps",
load_best_model_at_end=True,
warmup_steps=100,
lr_scheduler_type="cosine",
)

trainer = Trainer(
model=model,
args=training_args,
train_dataset=dataset["train"],
eval_dataset=dataset["test"],
tokenizer=tokenizer,
data_collator=transformers.DataCollatorForLanguageModeling(tokenizer, mlm=False),
)
```

Step 6: Train

```
trainer.train()
```

Step 7: Save and Inference

Save fine-tuned adapter

```
model.save_pretrained("./llama-dolly-adapter")
```

Load for inference

```
from peft import AutoPeftModelForCausalLM

model = AutoPeftModelForCausalLM.from_pretrained(
"./llama-dolly-adapter",
device_map="auto"
)
```

Generate

```
prompt = "Explain quantum computing:"
inputs = tokenizer(prompt, return_tensors="pt")
outputs = model.generate(
**inputs,
max_new_tokens=100,
temperature=0.7,
top_p=0.95,
)
print(tokenizer.decode(outputs[0]))
```

10.2 RLHF Fine-tuning with TRL

```
from trl import PPOTrainer, PPOConfig, AutoModelForCausalLMWithValueHead
from trl import reward_model_trainer
```

Load base model and reward model

```
model = AutoModelForCausalLMWithValueHead.from_pretrained("meta-llama/Llama-2-7b")
reward_model = AutoModelForSequenceClassification.from_pretrained("reward-model-path")
```

PPO Configuration

```
ppo_config = PPOConfig(
model_name="meta-llama/Llama-2-7b",
learning_rate=1.41e-5,
batch_size=32,
mini_batch_size=4,
ppo_epochs=4,
)
```

Dataset

```
dataset = load_dataset("your-dataset")
```

Trainer

```
ppo_trainer = PPOTrainer(
model=model,
config=ppo_config,
train_dataset=dataset["train"],
tokenizer=tokenizer,
)
```

Training loop

for epoch in range(ppo_config.ppo_epochs):
for batch in ppo_trainer.dataloader:
# Generate responses
response_tensors = model.generate(batch["input_ids"])

```
    # Get rewards
    rewards = [reward_model.score(r) for r in response_tensors]

    # PPO step
    stats = ppo_trainer.step(batch["input_ids"], response_tensors, rewards)
```

10.3 Evaluation and Monitoring

from rouge import Rouge
from nltk.translate.bleu_score import sentence_bleu

Evaluation metrics

def evaluate_model(model, test_dataset):
rouge = Rouge()
bleu_scores = []

```
for example in test_dataset:
    prompt = example["instruction"]
    reference = example["response"]

    # Generate
    inputs = tokenizer(prompt, return_tensors="pt")
    output = model.generate(**inputs, max_new_tokens=100)
    generated = tokenizer.decode(output[0])

    # Compute metrics
    rouge_scores = rouge.get_scores(generated, reference)
    bleu = sentence_bleu([reference.split()], generated.split())

    bleu_scores.append(bleu)

return {
```

```
    "avg_bleu": sum(bleu_scores) / len(bleu_scores),
    "avg_rouge": sum(r["rouge1"]["f"] for r in rouge_scores) / len(rouge_scores),
}
```

Evaluate

metrics = evaluate_model(model, test_dataset)
print(f"BLEU: {metrics['avg_bleu']:.4f}, ROUGE: {metrics['avg_rouge']:.4f}")

10.4 Production Deployment with vLLM

from vllm import LLM, SamplingParams
import json

Load model

llm = LLM(
model="meta-llama/Llama-2-7b",
tensor_parallel_size=1,
gpu_memory_utilization=0.9,
dtype="bfloat16",
)

Define sampling

sampling_params = SamplingParams(
temperature=0.8,
top_p=0.95,
max_tokens=512,
repetition_penalty=1.1,
)

API server

from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class GenerateRequest(BaseModel):
prompt: str
max_tokens: int = 512

@app.post("/generate")
async def generate(request: GenerateRequest):
outputs = llm.generate([request.prompt], sampling_params)
return {"response": outputs[0].outputs[0].text}

Run: uvicorn app:app --host 0.0.0.0 --port 8000

Conclusion and Future Directions

Generative AI with LLMs represents the frontier of machine learning and artificial intelligence. This module has covered the complete journey from understanding foundational architectures to deploying production systems. Key insights include:

Key Takeaways:

- Transformers and attention mechanisms enable powerful language models
- Scaling laws provide predictability for large-scale training
- Pre-training on diverse data creates general knowledge
- Fine-tuning and PEFT enable efficient task specialization
- RLHF aligns models with human values and preferences
- Deployment requires careful optimization for latency and cost

Future Directions:

- **Mixture of Experts:** Sparse models with higher capacity and efficiency
- **Multimodal Models:** Unified models for text, image, audio, and video
- **Long Context:** Extending context windows to million+ tokens
- **Reasoning at Scale:** Test-time compute for complex problem-solving
- **Efficient Inference:** Continued optimization for mobile and edge
- **Customization:** Personal models fine-tuned on individual data
- **Safety and Alignment:** Advancing techniques for responsible AI
- **Data Efficiency:** Learning from fewer, higher-quality examples

The field evolves rapidly, with new architectures, techniques, and applications emerging continuously. Practitioners must stay current with research, maintain hands-on expertise with tools, and consider both technical and ethical implications of their work.

References

[1] Brown, T. M., et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877-1901.

[2] Ouyang, L., et al. (2022). Training language models to follow instructions with human feedback. *arXiv preprint arXiv:2203.02155*.

[3] Touvron, H., et al. (2023). Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.

[4] Bisk, Y., Holtzman, A., Thomason, J., Andreas, J., Buys, J., Hawkins, R. D., ... & Turian, J. (2020). Experience grounds language. *arXiv preprint arXiv:2004.10151*.

[5] Vaswani, A., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.

[6] Orvieto, A., et al. (2023). Resurrecting recurrent neural networks for long sequences. *International Conference on Machine Learning* (pp. 26670-26697).

[7] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*.

[8] Wei, J., et al. (2022). Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*.

[9] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichien, B., Xia, F., ... & Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903*.

[10] Hoffmann, J., et al. (2022). Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*.

[11] Gao, Y., et al. (2023). The lifecycle of generative AI projects: A taxonomy and research roadmap. *arXiv preprint arXiv:2304.03854*.

[12] Kaplan, J., et al. (2020). Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*.

[13] Chinchilla Scaling: Research demonstrates compute-optimal scaling with proportional increase in model and data size. DeepMind research, 2022.

[14] Wei, J., et al. (2021). Finetuned language models are zero-shot learners. *International Conference on Learning Representations*.

[15] Malladi, V. S., et al. (2023). Fine-tuning language models for factuality. *arXiv preprint arXiv:2311.08401*.

[16] Lv, K., Yang, Y., Liu, T., Gao, Q., Guo, Q., & Qiu, X. (2023). Full parameter fine-tuning for large language models with limited computational resources. *arXiv preprint arXiv:2306.07629*.

[17] Hu, E. J., et al. (2021). LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*.

[18] Dettmers, T., et al. (2023). QLoRA: Efficient finetuning of quantized LLMs. *arXiv preprint arXiv:2305.14314*.

[19] Christiano, P. F., et al. (2023). Deep reinforcement learning from human preferences. *Advances in Neural Information Processing Systems*, 29.

[20] Goldberg, Y., et al. (2023). Eleuther AI's RLHF implementation and analysis. GitHub Repository.

[21] Rafailov, R., et al. (2023). Direct preference optimization: Your language model is secretly a reward model. *arXiv preprint arXiv:2305.18290*.

[22] Bai, Y., et al. (2022). Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*.

[23] Maynez, J., Narayan, S., Artetxe, M., & Schwenk, H. (2020). On faithfulness and factuality in abstractive summarization. *arXiv preprint arXiv:2005.00661*.

[24] Mirzadeh, I., et al. (2024). Fast LLM inference with speculative decoding. *arXiv preprint arXiv:2302.01318*.